Web Systems Development

Using AI on the project: you remain the engineer

Session 23 · Tue 11/17 · twenty minutes · Claude Code, ChatGPT and Cursor, and one way to think about all three

RPI

THE ELEPHANT IN THE ROOM

Meet the 900-pound pair programmer

AI is already in the project. The awkward part is pretending “use it responsibly” is a workflow.



WHAT EVERYONE SAYS

“AI is just a tool.”

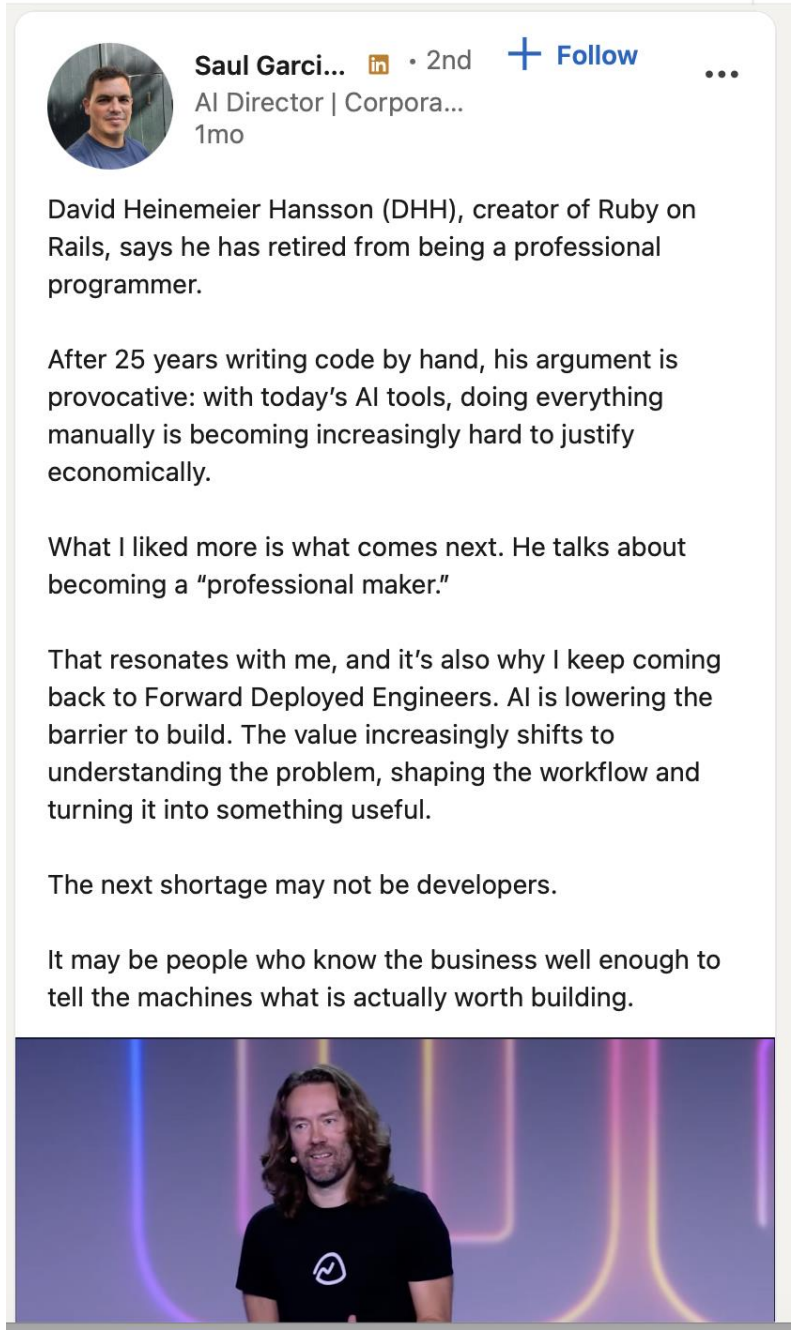
WHAT THE ELEPHANT SAYS

```
Done. Two files
changed.
Want me to commit?
```

Ignoring it is not a policy. Review is.

Welcome to the Age of AI.

https://www.linkedin.com/posts/saulgh_david-heinemeier-hansson-dhh-creator-of-ugcPost-7509554736849678336-JUqy/



Spec → Delegate → Verify → Own, mapped to the project

SPEC

What exactly do I want, and how will I know it is right?

IN THE PROJECT

- API.md: fields, validation, who may do what, error shapes
- The lab check scripts
- A failing test, written first

DELEGATE

Which tool, with what context, for how big a step?

IN THE PROJECT

- Pick the tool by how much autonomy the task deserves
- Give it the repo's AGENTS.md
- One endpoint, one component, one migration at a time

VERIFY

Does it do what the spec says, and nothing else?

IN THE PROJECT

- docker compose run --rm unit · api · e2e
- Read the whole diff before you commit
- The PR review a teammate already owes you

OWN

Could I explain this line to the grader? Would I have written it this way?

IN THE PROJECT

- The oral-check rule
- The AI Use Statement
- One line in this week's PROCESS.md

Every step is something the project already asks for. The AI workflow is not paperwork beside the project; it is the project, used in order.

Build the minimum, then a human looks. Five times.

1 · WIREFRAMES

AI GENERATES

Three versions of the five pages, not one. Versions are cheap here.

A HUMAN REVIEWS

The team picks one; two people from the audience react to it.

"Would a person open this twice?"

2 · REACT, FAKE DATA

AI GENERATES

The midterm shape: pages, rules.js, validation messages. One page at a time.

A HUMAN REVIEWS

Teammates click through every page; the midterm appendix is the checklist.

Five pages work on fake data.

3 · API.md

AI GENERATES

Options only: field types, validation rules, error shapes, what needs admin.

A HUMAN REVIEWS

Humans write the contract and defend every line in the lab walkthrough.

The API design review.

4 · BACKEND

AI GENERATES

One endpoint at a time, against its API test.

A HUMAN REVIEWS

PR review, plus the three suites.

compose run --rm api is green.

5 · BEYOND

AI GENERATES

Only after the floor is deployed.

A HUMAN REVIEWS

The report's evidence: a number, a test, a user.

The grader can run it.

Why this order saves tokens: a wrong assumption caught at the wireframe costs one prompt; caught after the backend it costs a rewrite of three layers. The human look before each stage is the cheapest token you will spend.

Versions are cheap at the start and expensive at the end: ask for three wireframes and one backend. Generating alternatives is what the tool is best at; choosing is what the team is for.

Same request, one difference: it was given API.md

PREDICT One of these would fail at least three of your API tests. Which one, and which tests?

WITHOUT THE SPEC

```
> Write the Laravel update endpoint for a listing.

function update(Request $r, Listing $l) {
  $l->update($r->all());
  $l->status = $r->input('status', 'active');
  $l->save();
  return response()->json($l);
}
```

WITH API.md PASTED IN

```
> Here is API.md. Write PUT /api/listings/{id} to spec.

function update(UpdateListingRequest $r, Listing $l) {
  $this->authorize('update', $l);  // owner or admin
  $l->update($r->validated());     // title, body, price
  return response()->json(['data' => $l]);
}
```

Left, four bugs the suite catches: mass-assigns everything the client sends (user_id? role?), invents a status field the spec does not have, skips the 403 for editing someone else's listing, and returns the wrong shape (no data wrapper).
Right: the same model, told what the spec says. The tool did not get smarter; it got the spec. API.md is the prompt you were already required to write.

AGENTS.md: a note to a colleague, in your voice

```
# Team 3 — Study Buddy

## Who you are working with
I am a student in a web development class, building this app with
a team of four. I want to fully understand the platform I am
building, not just ship it: I will be asked to explain any line of
it in person. So teach as you go: say what each piece does and why,
name the Laravel or React idea behind it, and prefer the simple
version I can follow over the clever one.

## Where to look first
- context/ has the background: the course project description, the
  midterm appendix, the wireframe we chose, my notes on this week's
  readings, and how we build things in this class. Read it before
  suggesting code, and tell me what is missing from it.
- API.md at the root is the contract. If it looks wrong, say so.
- Laravel 13 in api/, React + Vite + Tailwind in web/, MySQL, Docker.

## How to hand me work
- Write new code to scratch/<task>/ with a NOTES.md: what each file
  is, where it goes, and why. I move it into api/ or web/ myself,
  so that I read every line.
- One endpoint or one component at a time, with its test.
- Before you say it is done, run the suite that covers it:
  docker compose run --rm unit | api | e2e. The app runs under
  docker compose watch; nothing is installed on the host.
```

WHAT IT IS

A file in the repo root that every tool reads before it acts (Codex and Cursor read AGENTS.md; Claude Code reads CLAUDE.md, which can be one line, @AGENTS.md). Three things, in your words: who I am and what I am trying to learn, where to look, how to hand work back. Not a list of rules.

THE CONTEXT/ FOLDER

The second half of the idea. Put in it what the tool cannot see in the code: the project description, the midterm appendix, the wireframe you chose, your reading notes, and how we build things here. It stops guessing, and your notes come back in your own vocabulary.

FRIDAY

First ten minutes of the lab: write yours from the full version in project/AI_PROMPTS.md, make context/, add scratch/ to .gitignore, open it as a PR, one teammate reviews it.

AI output goes to scratch/, never straight into the project

```
> Build the "create listing" page. Put everything in
> scratch/create-listing/ and write NOTES.md explaining
> each file and where it goes in web/. Do not edit web/.

scratch/create-listing/
  NOTES.md                what each file is, where it
                          moves, and why
  CreateListing.jsx       -> web/src/pages/
  CreateListing.test.jsx  -> web/src/pages/
  api.js.patch            two functions to add to
                          web/src/api.js
  App.jsx.patch           the route to add

# then: you move each file in by hand, run the tests,
# and the PR is what you moved, not what it emitted.
```

THE RULE

A prompt never implements a feature straight into api/ or web/. It writes to scratch/<task>/ (gitignored) with a NOTES.md. You move it into the project by hand, file by file, reading every line on the way. Exception: a one-line fix you type yourself.

WHY

Moving is reading: the oral check is passed during the move, not after. The diff is a human's change, so a teammate can review it. The tool cannot break the running app or the branch. And NOTES.md plus your commit message is the AI Use Statement, written for free.

Say it in AGENTS.md so every tool works this way, and put scratch/ in .gitignore so it never reaches main. The folder is yours to delete when the move is done.

The tool writes the code.
You sign it.

Spec it, delegate it, verify it, own it. Build the minimum, let a human look, then the next stage. Draft in scratch/, carry it in by hand. The grader will ask you, not the tool, why line 14 is there.